How Often Will My Agent Overspend? Certified Run-Level Budget Decisions for LLM Agents

Robat Das
Independent Researcher
ORCID: 0009-0006-7792-8351

November 2026

Abstract

A budget enforcer for an LLM agent must decide before every call whether it is affordable, without knowing how long the response will be. Reserving the expected output gives no control over how often a run overspends; reserving the maximum output never overspends but can make capable models unaffordable. We certify the guarantee in between, that a whole run exceeds its budget with probability at most ε (with confidence $1-\delta$), by testing a family of reservation policies in a fixed sequence with exact binomial p-values (Learn-then-Test). It needs only i.i.d. runs, and for admit/refuse controllers one batch of at least $\ln(1/\delta)/\varepsilon$ logged runs serves every policy. Per-call risk levels do not translate into run-level risk, and per-step union bounds forfeit most of the capability a budget can buy. In simulation, 320,000 calibrations scored against precisely known risks selected an unsafe policy in at most 5.3% of cases in any setting (95% CI 4.4–6.4%), consistent with $\delta = 5\%$. On 2,100 real gpt-4o-mini and gpt-4o runs replayed under the shipped controller, no setting showed a significant violation; gains over certified expected-cost reservation are small at $\varepsilon = 0.05$ and large at tight tolerances. On 3,000 long-horizon coding-agent trajectories overspending is negligible, but runs that cannot finish within budget absorb up to 61% of spend. For such agents we certify a second decision, stopping a run early: with at most 5% of all runs stopped although they would have solved their task, the certified rule saved 11–23% of spend, while an uncertified threshold was unsafe at tight budgets. The price of one maximum-length output relative to the budget predicts which decision matters. Both methods ship in the open-source library Baar-Core.

1 Introduction

An LLM agent alternates between model calls and tool calls until it produces an answer. A call to model m costs $p_m^{\text{in}} n_t + p_m^{\text{out}} y_t$, where n_t is the prompt length, which grows as the conversation grows, and y_t the output length. Only n_t is known before the call. Repeated runs of the same agentic task can differ by up to 30× in tokens, and agents predict their own consumption poorly [5, 13]. Unbounded consumption is a recognised risk for LLM applications [17], and dozens of production overrun incidents have been catalogued in agent frameworks [12].

A budget enforcer decides, before each call, whether the call is affordable, and so must plan for an unknown output length. Reserving the *expected* output is permissive but gives no control over how often a run overspends. Reserving the provider's *maximum* output never overspends, but when one maximum-length output of the capable model costs a large share of the budget, that model is rarely admitted. We study the guarantee in between. Given a run budget B, a tolerance ε and a confidence level $1-\delta$, deploy a controller π such that, with probability at least $1-\delta$ over

calibration,

$$R(\pi) \;=\; \Pr_{\text{task, sampling}}\big(\text{cost of a run under } \pi > B\big) \;\le\; \varepsilon. \tag{1}$$

Two natural alternatives fall short. Calibrating each *call*, so that a response exceeds its reservation with probability at most some level, is the approach of conformal cost gates [7, 20], but a per-call level says little about (1): whether a long response causes an overrun depends on how much budget is left, and a run contains many calls (Section 5). Splitting ε across the steps of a run with a union bound is valid but charges every step for a tail event that rarely matters (Section 5).

Contributions.

1. **Run-level certification.** We certify (1) by testing a fixed family of reservation policies in sequence with exact binomial p-values, an instance of Learn-then-Test [2]. The statistical tool is not new. We make explicit what the budget setting needs: the guarantee holds with calibration runs shared across policies, so for admit/refuse controllers one batch of logged runs can be replayed under every policy; no policy can be certified with fewer than $\ln(1/\delta)/\varepsilon$ runs; and the fallback policy must have zero risk, which holds for worst-case reservation only if prompt counts are upper bounds (Section 4).

2. **Why per-call and per-step control fall short**, with two mechanisms, sample size and tail cost, and a characterisation of when output reservation matters at all: the ratio κ of one maximum-length output's price to the run budget (Section 5).

3. **Evaluation at three scales**: a simulator with 200,000-task pools per setting, so that the risk of every selected policy is known precisely (Section 6); 2,100 real short-horizon runs replayed exactly under the shipped controller with held-out splits (Section 7); and 3,000 real long-horizon coding-agent trajectories (Section 8), where we find that overspending is not the binding risk at all.

4. **Certified early stopping.** For long-horizon agents, where overspending is rare but runs that cannot finish absorb much of the budget, we certify a rule that stops runs at checkpoints so that at most ε of all runs are stopped although they would have succeeded within budget, and evaluate it on the coding-agent trajectories (Section 9).

5. **Software.** The method is the `calibrated` mode of the open-source library Baar-Core (v0.11.0). The real-model study also exposed a prompt-counting bug in the library, now fixed (Section 10).

2 Related work

Cost-aware routing and cascades. FrugalGPT [9] and RouteLLM [15] trade cost against quality across models, evaluated by average cost. C3PO [20] uses conformal prediction to bound the probability that the cost of answering *one query* with a cascade exceeds a budget. Treating a whole agent run as the query would give a run-level bound, but a cascade's cost is a fixed function of the query, while an agent's run cost depends on the controller's own admission decisions; our calibration is over that controller.

Budget enforcement for agents. Khan [12] catalogues 63 overrun incidents and enforces a hard dollar cap with an affine-typed Rust library, reporting 4–6× static over-reservation. Green SARC [7] places enforcement gates in the agent loop, calibrates per-action cost forecasts with split-conformal prediction, and reports zero over-budget incidence for its architectural gate. These systems make the cap hard by reserving conservatively, or calibrate individual actions; we are not aware of work that certifies the probability that an adaptive run overspends while letting the operator trade that probability for capability.

Distribution-free risk control. Conformal prediction and risk control [3, 6] bound the expected loss of a calibrated predictor; Learn-then-Test [2] recasts the choice of a hyperparameter as multiple testing and controls non-monotone risks. Anytime-valid variants handle sequentially arriving data [11]; ToolChain-CRC applies trajectory-level risk control to agent correctness under drift [16]; DAPRO [10] allocates evaluation compute across multi-turn interactions, meeting its budget in expectation. Learn-then-Test has been used to calibrate early exit inside a single generation [18]; BAGEN trains agents to stop early on trajectories that are unlikely to succeed, saving 28–64% of tokens on failed trajectories, without a guarantee on the successful runs it stops [13]. We certify the stopping rule of a whole agent run. The scenario approach [8] gives related finite-sample guarantees for decisions chosen from sampled constraints, and constrained MDPs [1] formalise expected-cost constraints in sequential decisions. Our setting is a chance constraint on a run's total cost under a fixed, adaptive controller.

3 Problem setup

A task X is drawn i.i.d. from a distribution $\mathcal{D}$. A run proceeds in steps $t = 1, 2, \ldots$. Before step t the controller knows the amount spent so far s_{t-1}, an estimate $\hat{n}_t$ of the prompt length, and the history. It chooses a model m_t, or refuses, which ends the run without an answer. After the call it observes y_t and pays $c_t = p^{\text{in}}_{m_t} n_t + p^{\text{out}}_{m_t} y_t$.

Reservation policies. A call to model m is admitted iff $s_{t-1} + p^{\text{in}}_m \lceil (1+\mu)\hat{n}_t \rceil + p^{\text{out}}_m u_m \le B$, where u_m is the planned output length and $\mu \ge 0$ a prompt margin. The plan defines the policy: **estimate** sets u_m to the mean logged output length ($\mu = 0$); **preflight** sets $u_m = Y^{\max}_m$, the provider's maximum output; **calibrated** sets $u_m = \hat{q}_\beta(m)$, the empirical $(1-\beta)$ quantile of logged output lengths of model m, with β chosen by calibration. A calibrated policy at level β is exactly per-call quantile reservation at level β; what differs is how β is chosen.

Modes and risk. In **Mode A** responses are never cut off and the run loss is $L = \mathbf{1}\{\sum_t c_t > B\}$, summed over the calls that were made, so a run that is refused after an admitted call has already crossed B counts as an overrun. In **Mode B** each call carries a `max_tokens` equal to the largest affordable output, so overrun is impossible and $L = \mathbf{1}\{\text{some response was cut off}\}$. Mode A is the default in Baar-Core, because a silently truncated answer can corrupt downstream results. The risk of a policy is $R(\pi) = \mathbb{E}[L(\pi)]$.

When does output reservation matter? Let $\kappa_m = p^{\text{out}}_m Y^{\max}_m / B$ be the price of one maximum-length output of model m as a fraction of the run budget. If $\kappa_m \ge 1$, preflight can never admit m; if $\kappa_m \ll 1$, the difference between reservation policies is a small share of the budget, and which policy is used hardly matters. The three workloads below span κ from 0.013 to about 70.

4 Run-level certification

Fix in advance a sequence of policies $\pi_1, \ldots, \pi_J$, ordered from conservative to aggressive, and a fallback π_0. For us the family is the calibrated policy at each level β_j together with estimate mode, ordered by the number of output tokens they reserve, largest first; any fixed order is valid, and this one tests the most conservative plans first. The quantile tables that define the π_j are estimated on data disjoint from calibration, so each $R_j = R(\pi_j)$ is a fixed number.

Assumption 1. $R(\pi_0) = 0$. *Refusing every run satisfies this trivially. Preflight satisfies it in Mode A if every call's reserved cost is at least its billed cost, that is, if* $p^{in}\lceil(1+\mu)\hat{n}_t\rceil \geq p^{in} n_t$ *and outputs never exceed* $Y^{\max}$*; and in Mode B unconditionally.*

Procedure. Draw n i.i.d. calibration runs. For each j let F_j be the number of runs whose loss under π_j is 1; the same runs may be used for every j. With $p_j = \Pr(\mathrm{Bin}(n, \varepsilon) \leq F_j)$, test $j = 1, 2, \ldots$ in order, continue while $p_j \leq \delta$, and deploy $\pi_{\hat{j}}$ for the last j with $p_j \leq \delta$, or π_0 if $p_1 > \delta$.

Theorem 1. *Under Assumption 1, if calibration runs are i.i.d. draws of (task, sampling randomness) from the deployment distribution, then* $\Pr\big(R(\pi_{\hat{j}}) > \varepsilon\big) \leq \delta$*, for any output-length distribution, any dependence between steps, any dependence between the* F_j*, and without monotonicity of* R_j *in* j*.*

Proof. If $R_j \leq \varepsilon$ for all j, every possible deployment has risk at most ε. Otherwise let j^* be the smallest index with $R_{j^*} > \varepsilon$; it is deterministic. Deploying a policy with risk above ε requires $\hat{j} \geq j^*$, hence $p_{j^*} \leq \delta$. Marginally $F_{j^*} \sim \mathrm{Bin}(n, R_{j^*})$, which is stochastically larger than $\mathrm{Bin}(n, \varepsilon)$, so $\Pr(p_{j^*} \leq \delta) \leq \delta$. Only the marginal law of F_{j^*} is used, so sharing runs across j is allowed. □

This is the fixed-sequence instance of Learn-then-Test [2]. Three consequences matter in practice.

Calibration cost. For a controller that only admits or refuses calls, refusing never changes the steps before it. One batch of n runs logged without a binding budget can therefore be replayed exactly under every π_j, and calibration costs n runs rather than $(\hat{j}+1)n$. For controllers whose policy changes the trajectory (for example by routing to a different model), each tested level needs its own runs, at most $\min(\hat{j}+1, J)\, n$.

Proposition 1. *No policy can be certified unless* $(1-\varepsilon)^n \leq \delta$*, i.e.* $n \geq \ln(1/\delta)/(-\ln(1-\varepsilon)) \approx \ln(1/\delta)/\varepsilon$*. At* $\delta = 0.05$ *this is 299 runs for* $\varepsilon = 0.01$*, 149 for* 0.02*, and 59 for* 0.05*.*

Proof. $p_1 \geq \Pr(\mathrm{Bin}(n, \varepsilon) \leq 0) = (1-\varepsilon)^n$. □

What the tables do and do not affect. The guarantee does not depend on how good the quantile tables are, only on their being fixed before calibration; tables fitted on the calibration runs would make the π_j data-dependent. Poor tables can only make the family less useful: every certified policy is still safe, but it may buy less capability.

5 Why per-call and per-step control fall short

Per-call levels. Reserving the $(1-\beta)$ quantile bounds, for exchangeable calls, the probability that one response exceeds its reservation by β. The run-level risk of the same policy can be far

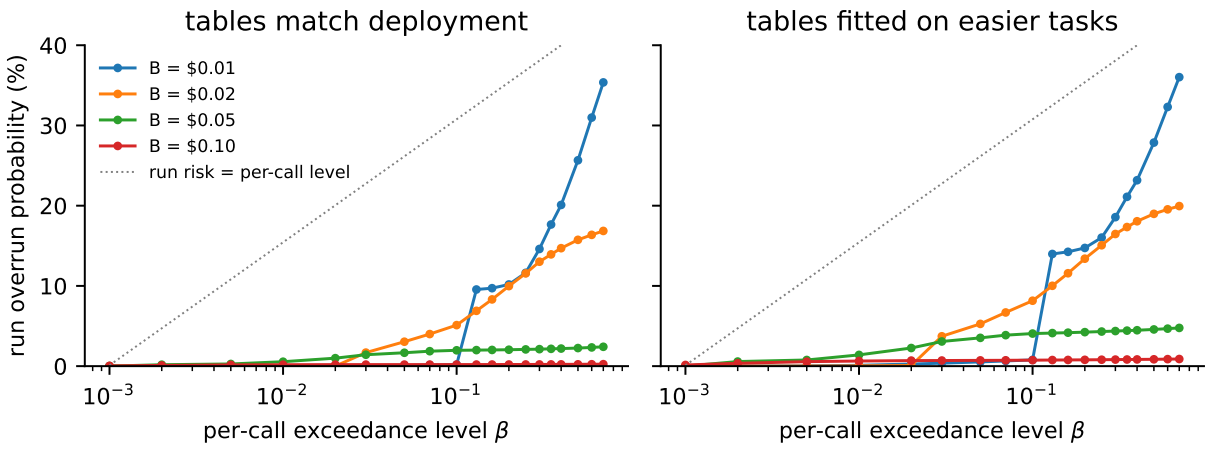


Figure 1: Run-level overrun probability of per-call quantile reservation at level β, in the simulator (Mode A). Each curve is one run budget; risks are measured on 100,000 held-out tasks. The dotted line is run risk = β.

lower or higher. It is lower when most runs end with budget to spare, so an exceeding response rarely matters: in our simulator, per-call level $\beta = 0.3$ gives run risk between 0.2% and 18.6% depending only on the budget and the tables (Figure 1). It is higher when budgets are tight and runs are long: if each of T calls has independent exceedance probability β and any exceedance overruns, the run risk is $1 - (1 - \beta)^T$. The mapping from β to run risk depends on the workload and the budget, so it has to be measured at the run level.

Per-step union bounds. The direct route to (1) through individual calls chooses levels α_t with $\sum_t \alpha_t \le \varepsilon$ and reserves an upper confidence bound on each $(1-\alpha_t)$ quantile; if no response exceeds its reservation, the run cannot overrun. Our baseline spends a fraction of the remaining ε only when a step is tight (risk spending, half per tight step, split between the step and a wrap-up reserve, so the first tight step gets $\alpha_1 = \varepsilon/4$), with exact binomial quantile bounds, Bonferroni-corrected across 4 strata and 40 quantile levels. It fails for two reasons.

Proposition 2. *With n i.i.d. samples, the smallest order statistic that upper-bounds the $(1-\alpha)$ quantile with confidence $1-\delta'$ is $Y_{(j)}$ for the least j with $\Pr(\mathrm{Bin}(n, 1-\alpha) \ge j) \le \delta'$; none exists unless $n \ge \ln(1/\delta')/(-\ln(1-\alpha))$.*

With $\delta' = 0.05/160$, the first tight step uses the largest grid level not above $\varepsilon/4$, which is 0.00213 at $\varepsilon = 0.01$; that requires 3,790 samples per stratum, more than the 2,000 we log, so the bound falls back to $Y^{\max}$ (*sample size*). From $\varepsilon = 0.02$ (1,966 samples) the bounds are finite but sit far in the tail, and one capable-model call priced at that tail costs more than the remaining budget (*tail cost*). Both mechanisms come from charging each step for its own tail, while a run-level overrun needs a long response *and* a run that is already near its budget.

6 Simulation study

Simulator. Two models are priced at \$0.15/\$0.60 (small) and \$2.50/\$10.00 (capable) per million input/output tokens, with maximum outputs of 4,096 and 8,192 tokens. Output lengths are

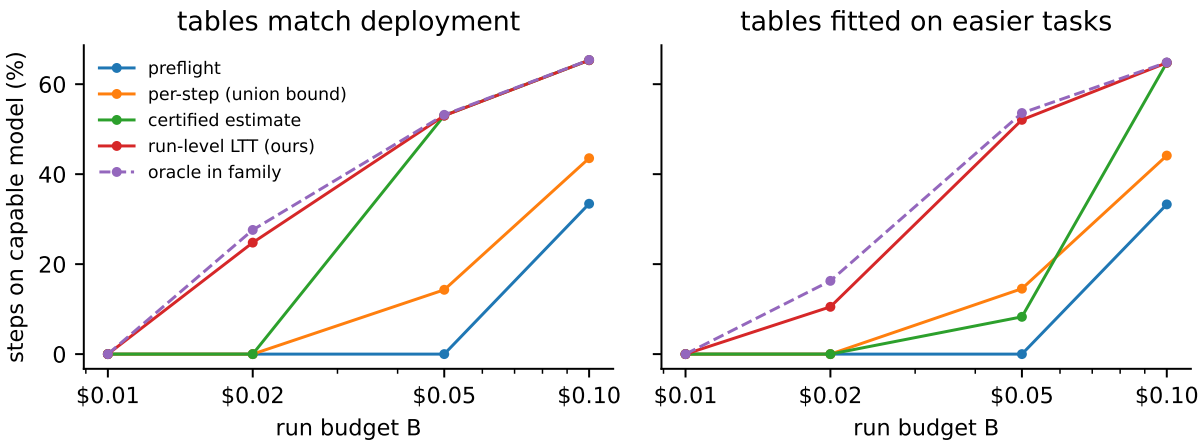


Figure 2: Share of steps on the capable model at $\varepsilon = 0.05$, $n = 1{,}000$, across run budgets (Mode A).

lognormal per (model, step type), longer for the capable model and for final answers. A run starts with a 600-token prompt; each tool step appends its output and an observation of 50–800 tokens. A task needs $1 + G$ steps with $G \sim \text{Geometric}(1/6)$ (mean 7), capped at 30. The controller prefers the capable model while more than half the budget remains, downgrades to the small model when the capable one does not fit, admits a tool step only if a later small-model final answer also fits, and otherwise wraps up early. This stylised controller is richer than Baar-Core's admission rule; the real-trace studies below use the shipped rule. Quantile tables (linearly interpolated; the real-trace studies use the library's order statistics) and the estimate-mode means come from 2,000 logged outputs per stratum.

Scenarios. In the **matched** scenario the tables come from the deployment distribution. In the **shifted** scenario the tables and the estimate-mode means are fitted on easy tasks, but in deployment every task has a hidden difficulty $d \sim \mathcal{N}(0, 0.6^2)$ that scales all its output lengths by e^d, so the tables are wrong in both mean and tail, and steps within a run are correlated.

Protocol. For each scenario, budget $B \in \{0.01, 0.02, 0.05, 0.10\}$ (Mode A) and $B = 0.02$ (Mode B), we draw 200,000 tasks and run each under preflight, estimate and the calibrated policy at 19 levels $\beta \in [0.001, 0.7]$, with common random numbers. For each $\varepsilon \in \{0.01, 0.02, 0.05, 0.10\}$ and $n \in \{100, 300, 1{,}000, 3{,}000\}$ we draw 2,000 calibration sets of n tasks without replacement from 100,000 of these tasks (close to i.i.d. sampling at these n) and run the procedure with $\delta = 0.05$. Every selected policy is then scored against its *true* risk and outcomes, estimated on a fresh pool of 400,000 tasks per setting (1,000,000 for one) with disjoint seeds, so that a risk near 10% is known to within 0.05 percentage points. A *violation* is a selected policy whose true risk exceeds ε; it is *clear* if the excess is more than two standard errors. We compare against preflight; the per-step baseline (20,000 runs per setting); estimate mode, which has no guarantee; a *certified estimate* that runs one binomial test on estimate mode and falls back to preflight if it fails (Learn-then-Test with $J = 1$); and the *oracle*, the member of the family with the highest capable-model share among those whose true risk is at most ε.

Table 1: Simulator, matched scenario, Mode A, $\varepsilon = 0.05$, $n = 1{,}000$. κ: price of one maximum capable-model output over B. "Big": share of executed steps on the capable model (%). Risk in %. Violation: share of 2,000 calibrations whose selected policy has risk above ε.

B	κ	preflight	per-step	estimate	cert. est.	run-level LTT		oracle
		big	big	risk / big	big	risk / big	viol.	big
\$0.01	8.2	0.0	0.0	15.3 / 27.0	0.0	0.2 / 0.0	0.0	0.0
\$0.02	4.1	0.0	0.0	13.5 / 44.6	0.0	3.4 / 24.8	3.5	27.6
\$0.05	1.6	0.0	14.3	2.1 / 53.1	53.0	2.4 / 53.0	0.0	53.2
\$0.10	0.8	33.4	43.5	0.2 / 65.3	65.3	0.2 / 65.3	0.0	65.4

Shifted scenario (tables fitted on easier tasks), same settings.

B	κ	preflight	per-step	estimate	cert. est.	run-level LTT		oracle
		big	big	risk / big	big	risk / big	viol.	big
\$0.01	8.2	0.0	0.0	19.2 / 28.2	0.0	0.8 / 0.0	0.0	0.0
\$0.02	4.1	0.0	0.0	17.0 / 45.4	0.0	2.4 / 10.5	2.5	16.3
\$0.05	1.6	0.0	14.5	4.4 / 53.5	8.3	3.6 / 52.1	0.0	53.6
\$0.10	0.8	33.3	44.1	0.8 / 64.8	64.8	0.9 / 64.8	0.0	64.8

Table 2: Validity over all settings (2 scenarios $\times$ 5 budget/mode configurations $\times$ 4 values of ε), 2,000 calibrations each, $\delta = 0.05$, in %. Violation: selected policy's true risk above ε; clear: by more than two standard errors; calibration population: risk in the 100,000-task pool the calibration sets were drawn from. Fallback: share of calibrations that certified nothing.

n	settings	pooled viol.	max viol.	max clear	max (cal. pop.)	fallback
100	40	0.5	3.6	3.6	3.6	50.0
300	40	0.8	5.1	3.8	5.1	1.1
1,000	40	0.7	5.3	3.5	5.3	0.0
3,000	40	0.5	3.9	2.6	3.9	0.0

Results. Table 1 and Figure 2 show what each approach buys at $\varepsilon = 0.05$. Preflight and the per-step baseline use the capable model for no step while $\kappa \geq 4$; per-step control reaches 14% at $\kappa = 1.6$ and 44% at $\kappa = 0.8$, against 53% and 65% for run-level calibration. Uncertified estimate mode buys about as much capability, but its risk is whatever the workload makes it: at the two tightest budgets it overruns 13.5–15.3% of runs with matched tables and 17.0–19.2% with shifted ones, about three times the tolerance or more. The certified estimate is competitive only when estimate mode is far below ε; otherwise its single test fails and it falls back to preflight (no capable-model steps at $B = \$0.02$, and 8.3% in the shifted scenario at $B = \$0.05$), whereas the run-level policy keeps 25% and 11% at $B = \$0.02$ and 52% in the shifted scenario at $B = \$0.05$. At $B = \$0.01$ ($\kappa = 8.2$) no member of the family that uses the capable model has risk at most 5%, so the run-level policy, like the oracle, uses only the small model: at this budget safety costs capability. At $\varepsilon = 0.05$ and $n = 1{,}000$ the certified policy is within 5.8 points of the oracle in Mode A and 6.0 in Mode B. At other tolerances the median gap over all settings is below one point, but it reaches 14–21 points in a few settings (for example 21 points in the shifted scenario at $B = \$0.05$ and $\varepsilon = 0.01$), where 1,000 runs give little power to certify the levels whose risk lies just below ε. In Mode B at $B = \$0.02$, estimate mode truncates 13.5% of runs (matched) and 17.1% (shifted); at $\varepsilon = 0.05$ the certified policy truncates 3.4% and 2.3% on average.

Table 2 summarises validity. Pooled over the 40 settings, the selected policy's true risk exceeded ε in 0.5–0.8% of calibrations at each calibration size. The worst setting reached 5.3% (107 of 2,000; 95% Clopper–Pearson interval 4.4–6.4%), consistent with $\delta = 5\%$: in the shifted scenario in Mode B at $\varepsilon = 0.10$ the procedure often selected a level whose true risk is within two standard errors of ε, which is where the bound of Theorem 1 is tight, since the test at that level rejects with probability close to δ; these violations are ambiguous rather than clear. The largest share of clear violations in any setting was 3.8%. At $n = 100$, the settings with $\varepsilon \leq 0.02$ can certify nothing, as Proposition 1 predicts, and fall back to preflight; from $n = 300$ fallback is rare (1.1%).

7 Real short-horizon runs: HotpotQA

Data. A ReAct-style agent [23] answers HotpotQA distractor questions [22]. It sees the titles of the ten context paragraphs, may call `lookup(title)` to read one, and stops with a final answer or after eight steps (temperature 0.7; answers scored by token F1). Runs are short, 2.6 calls on average, so dependence between steps is only lightly exercised. We logged 1,500 runs with gpt-4o-mini and 600 with gpt-4o on the same shuffled tasks: every call's prompt tokens, the controller's pre-call prompt estimate, output tokens and cost. The total spend was $1.85, itself capped by Baar-Core.

Exact replay of the shipped controller. We replay each logged run under each policy with the admission rule of Baar-Core 0.11.0: one plan per model, pooling step types, because the controller does not know in advance whether a call will be a tool call or a final answer; quantiles from the library's `QuantileTable`; and the pre-call prompt estimate the controller actually logged. Every policy, including estimate mode, prices the prompt with the same margin $\mu = 0.2$; the shipped estimate mode uses no margin and a fixed expected output of 500 tokens by default, so giving it the margin and the logged mean output favours the baseline. Since refusing a call never changes the steps before it, the replay is exact. For each of 200 random splits, 25% of runs build the quantile table and set the budget (a percentile of their unconstrained cost, from the 50th to the 99th, so the budget is fixed before calibration), 37.5% calibrate (shared across levels) and 37.5% are held out. The family is the calibrated levels $\beta \in [0.001, 0.99]$ plus estimate mode, ordered by reserved tokens. We report held-out outcomes and the share of splits in which a one-sided binomial test finds the held-out overrun rate significantly above ε.

What the traces look like. Outputs are short: the 99th percentile is 57–65 tokens and the maximum 181, while $Y^{\max} = 16{,}384$, so $\kappa \approx 70$ for gpt-4o at the median budget. The logged prompt estimate under-counted in 94% of calls, by a median of 29 tokens (9%), because the library's token counter could not take tool schemas; recounting a sample of first-step prompts with the schema matched the billed count to within one token. The 20% prompt margin does not absorb this: in 38% of calls, mostly first steps where the schema is about a fifth of the prompt, the margined estimate was still short. This does not affect the guarantee for calibrated policies, whose overruns are counted in calibration whatever their cause, but it means Assumption 1 would fail for preflight with this counter in general; here preflight admitted no run, so its risk was zero.

Results. Across all 30 settings (two models, five budgets, three values of ε; Table 3 shows three budgets), the held-out overrun rate was significantly above ε in at most one split of 200 (0.5%), within the test's own false-alarm rate. Preflight finished no run at any budget, because one maximum-length output costs more than the whole budget. With the prompt margin, estimate

Table 3: HotpotQA, held-out results over 200 splits (%). Estimate mode (same prompt margin) has no guarantee. Sig. viol.: share of splits where the held-out overrun rate is significantly above ε (one-sided binomial test, level 0.05). Preflight finished no run at any of these budgets.

		estimate		cert. est.	run-level LTT				oracle
model	B	over / done	ε	done	over	over p95	done	sig. viol.	done
			0.01	0.0	0.0	0.0	0.0	0.0	41.8
4o	p50	0.5 / 37.8	0.02	6.9	0.3	1.3	29.7	0.0	45.6
			0.05	37.6	2.0	3.6	46.9	0.0	48.4
			0.01	0.0	0.0	0.0	0.0	0.0	68.6
4o	p75	0.2 / 62.1	0.02	37.3	0.3	0.9	60.0	0.0	69.6
			0.05	62.1	0.9	2.2	69.7	0.0	69.7
			0.01	0.0	0.0	0.0	0.0	0.0	87.7
4o	p90	0.1 / 85.4	0.02	66.5	0.1	0.4	85.8	0.0	87.7
			0.05	85.4	0.1	0.9	87.7	0.0	87.7
			0.01	2.3	0.2	0.9	38.9	0.5	45.5
4o-mini	p50	0.8 / 45.4	0.02	31.4	1.0	2.0	45.8	0.0	48.9
			0.05	45.4	2.5	3.4	50.7	0.0	50.7
			0.01	4.4	0.3	1.1	57.4	0.5	67.3
4o-mini	p75	0.8 / 66.6	0.02	51.8	0.9	1.8	68.3	0.0	71.3
			0.05	66.6	1.1	2.0	71.4	0.0	71.4
			0.01	76.8	0.1	0.4	87.3	0.0	89.0
4o-mini	p90	0.1 / 88.5	0.02	88.5	0.1	0.4	89.0	0.0	89.0
			0.05	88.5	0.1	0.4	89.0	0.0	89.0

mode is already fairly safe on this workload (at most 0.8% overrun), so at $\varepsilon = 0.05$ the certified estimate passes its test and the run-level policy gains little: at the median and 75th percentile budgets it finished 47–71% of runs against 38–67%, 1.07–1.25 times as many, and matched the oracle within 1.5 points. The advantage appears at tight tolerances, where a single test of estimate mode fails. At $\varepsilon = 0.02$ the run-level policy finished 30–68% of runs against 7–52%; at $\varepsilon = 0.01$ with gpt-4o-mini, 39–57% against 2–4%. With gpt-4o, 225 calibration runs are fewer than the 299 that Proposition 1 requires for $\varepsilon = 0.01$, so nothing was certified and the procedure fell back to preflight, as it should. The run-level policy is not always near the oracle: at $\varepsilon = 0.02$ and the median budget it finished 30% of gpt-4o runs against the oracle's 46%, because 225 runs give little power to certify the more aggressive levels.

8 Real long-horizon runs: coding agents

Data. We sampled 3,000 of the 67,074 public OpenHands [21] trajectories on SWE-rebench [4] released by Nebius [14], Trofimova et al. [19] (CC-BY-4.0), all generated by Qwen3-Coder-480B-A35B-Instruct. Each assistant turn is one call; prompt tokens are the full preceding history and output tokens the turn's content and tool-call arguments, counted with the o200k tokenizer, plus 3,121 tokens per call for the agent's tool definitions, which are sent with every call but are not part of the logged messages. We re-price all runs at gpt-4o list prices without prompt caching; costs are therefore counterfactual, but the token sequences are real. Runs average 64 steps (maximum 100, the agent's limit), cost $5.09 on average (the most expensive 3.9 times the median), and resolve

Table 4: Coding-agent trajectories, held-out results over 200 splits (%), $\varepsilon = 0.05$. Over / done for each policy. Unconstrained: share of runs that finish when no budget applies.

budget	B	κ	preflight	estimate	run-level LTT	unconstrained done
p50	\$4.42	0.037	0.0 / 46.4	0.0 / 49.3	0.0 / 49.3	90.8
p75	\$6.44	0.025	0.0 / 73.0	0.0 / 74.4	0.0 / 74.4	90.8
p90	\$8.84	0.019	0.0 / 86.5	0.0 / 87.0	0.0 / 87.0	90.9
p95	\$10.26	0.016	0.0 / 89.4	0.0 / 89.6	0.0 / 89.6	90.8
p99	\$12.77	0.013	0.0 / 90.7	0.0 / 90.7	0.0 / 90.7	90.8

the issue in 47% of cases.

Results. Here κ is 0.013–0.037: one maximum-length output costs at most 4% of the budget, and input tokens are 97% of cost. Overspending was negligible: at most 0.005% of held-out runs under any policy (fewer than one in 20,000, all at the 50th and 75th percentile budgets) and none under preflight; the certified policy finished as many runs as estimate mode to within 0.05 points, and preflight finished at most 2.9 points fewer runs (Table 4). Output reservation is not where the budget bites on this workload. What does bite is runs stopped mid-task. Their number is set almost entirely by where the budget sits: at a budget equal to the q-th percentile of run cost, $1 - q$ of runs cost more than the budget under any policy, and the reservation policy adds at most 3.2 points to the share refused (50–53% at the median budget, 25–27% at the 75th percentile, 10–11% at the 90th). What the budget does not show is how much is spent on those runs before they are stopped: 58–61% of all spend at the median budget, 35–37% at the 75th percentile and 18–19% at the 90th. Stopping such runs early, or not starting them, is a different decision; Section 9 certifies it with the same theorem.

9 Certified early stopping

The coding-agent runs show that, for long-horizon agents, the money a budget wastes goes into runs that cannot finish within it. The certification of Section 4 applies to any fixed family of policies with a zero-risk fallback, so we use it for a second decision: whether to continue a run.

Policy family. At checkpoints $k \in \{0, 5, 10, 15, 20, 30, 40, 50, 60, 80\}$ (completed calls; $k = 0$ is the start of the run), a score $s_k \in [0, 1]$ estimates the probability that the run's total cost will exceed B. Policy π_τ stops the run at the first checkpoint with $s_k \geq \tau$. The scores come from one logistic regression per checkpoint, fitted on a training split that is disjoint from calibration, on what the controller has seen so far: the size of the first prompt, spend, context size and output lengths, and counts of the agent's actions (shell commands, file edits, file views, explicit "think" steps, test runs, tool outputs containing errors, repeated identical commands, errors among the last five tool outputs, and whether the agent has edited a file and tested after editing). The family is $\tau \in \{0.999, 0.995, \ldots, 0.1\}$ (18 levels), tested from the most conservative; the fallback never stops, so Assumption 1 holds trivially.

Harm and benefit. The loss is 1 if the policy stops a run that would have *solved* the task within the budget (the run is resolved and its cost is at most B), or, in a second variant, a run that would have *finished* within the budget. Theorem 1 then bounds the probability, over calibration, that the deployed rule's harm rate exceeds ε. The harm rate is a share of *all* runs: at most ε of

Table 5: Certified early stopping on the coding-agent trajectories, held-out results over 200 splits (%). Harm: share of all held-out runs that were stopped although they would have solved (or finished) within B, the quantity bounded by ε; of good: the same runs as a share of those that would have solved (or finished); p95 over splits; sig.: share of splits where the held-out harm rate is significantly above ε. Saved: share of all spend saved. Doomed stopped: share of runs costing more than B that were stopped early. Uncertified: stop whenever the score reaches 0.5, harm counted for solving runs.

B	harm for	ε	harm	of good	p95	sig.	saved	doomed stopped
	solving	0.01	0.4	1.4	1.1	0.0	6.8	54.2
	solving	0.02	1.1	3.6	2.0	0.0	12.3	69.1
	solving	0.05	3.2	10.9	4.7	0.5	23.3	83.6
p50	finishing	0.01	0.4	0.8	1.0	0.0	4.9	45.7
	finishing	0.02	1.0	2.1	2.0	0.0	8.9	61.2
	finishing	0.05	2.9	5.9	4.5	1.0	16.1	75.5
	uncertified, solving		23.4	79.8			84.5	99.3
	solving	0.01	0.4	1.1	0.9	0.0	3.0	63.8
	solving	0.02	1.1	2.7	2.0	0.0	6.9	75.6
	solving	0.05	2.8	6.9	4.5	0.0	14.5	84.7
p75	finishing	0.01	0.4	0.6	1.0	0.5	1.2	51.8
	finishing	0.02	1.1	1.4	1.9	0.0	2.8	64.6
	finishing	0.05	3.3	4.4	4.5	0.0	8.0	77.5
	uncertified, solving		7.8	19.5			29.9	93.6
	solving	0.01	0.4	0.9	0.9	0.0	1.7	48.1
	solving	0.02	1.0	2.2	1.8	0.0	3.6	61.5
	solving	0.05	3.1	6.9	4.5	0.0	11.0	80.0
p90	finishing	0.01	0.4	0.5	0.9	0.0	0.8	35.1
	finishing	0.02	1.1	1.2	1.9	1.0	1.8	49.8
	finishing	0.05	3.0	3.4	4.4	0.0	4.5	65.5
	uncertified, solving		2.2	4.8			7.9	74.8

all runs are stopped although they would have succeeded. As a share of the runs that would have succeeded it is larger, by the inverse of their frequency (29–45% of runs solve their task within these budgets); certifying that conditional rate instead means calibrating on the succeeding runs only, with correspondingly fewer runs. The benefit is the share of all spend saved, relative to a controller that lets every run continue until the budget stops it (we take its spend on a run as $\min(\text{cost}, B)$; the real controller refuses the call that would cross B, so it spends at most one call less, and the largest single call is at most 5.5% of B in 99% of runs at the tightest budget). Stopping a run never changes its earlier steps, so the logged runs replay exactly and one calibration batch serves every τ. We use 200 random splits (25% training and budget, 37.5% calibration, 37.5% held out), budgets at the 50th, 75th and 90th percentile of training-split cost, and $\delta = 0.05$.

Results. The guarantee held: the mean held-out harm rate was below ε in every setting, its 95th percentile over splits exceeded ε by at most 0.07 points, and at most 1% of splits showed a significantly higher rate, within the test's own false-alarm rate (Table 5). At $\varepsilon = 0.05$ for solving runs, the certified rule saved 23% of all spend at the median budget, 14.5% at the 75th percentile and 11% at the 90th, stopping 80–85% of the runs that could not finish within budget; it stopped 7–11% of the runs that would have solved. An uncertified rule that stops whenever the predicted

Table 6: Left: how well each checkpoint's features predict that a run will cost more than B (held-out AUC, 75th-percentile budget, 20 splits), and the spend an oracle would save by stopping every such run at that checkpoint. Right: spend saved by the certified rule ($\varepsilon = 0.05$, solving runs) with each feature set.

k	base	+behaviour	+repository	oracle saving
0	0.55	0.55	0.61	35.3
5	0.59	0.59	0.62	34.7
10	0.59	0.59	0.62	33.8
15	0.62	0.65	0.64	32.5
20	0.69	0.71	0.67	31.0
30	0.76	0.77	0.71	27.2
40	0.73	0.79	0.73	22.6

B	base	+behaviour	+repository
p50	16.3	23.3	20.8
p75	11.5	14.5	13.1
p90	7.9	11.0	10.0

probability reaches 0.5 is not calibrated to any of these budgets: at the median and 75th percentile it stops 23% and 8% of all runs although they would have solved (80% and 20% of the solving runs), far above ε; at the 90th it happens to be safe but saves less than the certified rule (7.9% against 11%).

The savings are bounded by when a run's fate becomes predictable (Table 6). At the start of a run, the only signals are the task's prompt and its repository, and they barely predict cost: AUC 0.55, or 0.61 with a repository prior fitted on the training split. An oracle that stopped every doomed run at the start would save 35% of spend at the 75th-percentile budget, but the rules we could build on these features came nowhere close. Prediction becomes useful after 15–30 calls (AUC 0.65–0.77 with behaviour counts), by which time the oracle ceiling has fallen to 27–33%. Behaviour counts add 3–7 points of saving over cost features alone; the repository prior helps at the start but not later. Certified start-of-run admission therefore needs better predictors of task cost than the ones available in these logs; the certification itself carries over unchanged. (Table 6 uses 20 half-and-half splits, for a less noisy AUC; the other analyses use the 25/37.5/37.5 splits above.)

10 Implementation

The `calibrated` mode is part of Baar-Core 0.11.0, an open-source Python library that estimates the cost of each LLM call locally, reserves it atomically against a budget before any request is sent, and reconciles with the provider-reported cost afterwards. Calibrated mode is preflight with the reservation lowered to the certified quantile; responses are never cut off by the budget, and when nothing is certified it reserves the worst case.

```
table = QuantileTable.from_records(logged_calls)
res = ltt_select(betas, run_agent_batch, n=1000, eps=0.05, delta=0.05)
router = BAARRouter(budget=0.10, on_output_overrun="calibrated",
                    output_quantiles=table, calibration_beta=res)
```

The function `ltt_select` implements Section 4 with exact binomial p-values computed in log space; for replayed runs, `calibrate_from_failures` takes the failure counts directly. The library's existing `overrun_risk` option is per-call quantile reservation at a fixed level; calibrated mode chooses that level at the run level. The HotpotQA study exposed that the library's prompt counter ignored tool schemas, which providers bill as prompt tokens, and that its OpenAI-compatible proxy dropped tool definitions; both are fixed in 0.11.0.

Certified early stopping (Section 9) is part of Baar-Core 0.12.1. `fit_early_stop` fits the per-checkpoint models on logged runs, `calibrate_early_stop` certifies the threshold by replaying calibration runs, and the router checks the policy before every call, ending the run with a `RunStopped` exception before any request is sent. Integrations report behaviour with `record_event` (for example a tool error).

```
policy = fit_early_stop(train_runs, budget=5.0, counter_keys=keys)
policy = calibrate_early_stop(policy, calib_runs, eps=0.05, delta=0.05,
                              harm="resolve")
router = BAARRouter(budget=5.0, early_stop=policy)
```

The library is dependency-free; on the logged coding-agent data its features are identical to those of the analysis code and its fitted models agree with it to within 10^{-11}. In deployment, the context-size feature comes from the router's own prompt estimate, so models should be fitted on logs produced the same way, and each router should serve one run.

Release 0.13.0 closes the loop from logs to deployment. Given a path for a run log, the router records whole runs with the pre-call prompt estimate, the billed cost and per-token prices of every call; the command `baar-calibrate` certifies either decision from such a log, replaying runs exactly as in Section 7; and `GuaranteeMonitor` watches a certified reservation level in deployment with an anytime-valid sequential test, warning when live runs overspend more often than ε, as they will if the workload has drifted from the calibration runs.

11 Limitations

- **Exchangeability.** The guarantee assumes deployment runs come from the calibration distribution. Price changes, model updates or a new task mix require recalibration; anytime-valid monitoring [11] is a natural complement.
- **Fallback.** Assumption 1 holds for preflight only if prompt estimates, after the margin, are upper bounds. With the pre-fix token counter they were not in 38% of HotpotQA calls; with tool schemas counted (Baar-Core 0.11.0) the estimate matched billed counts in our checks, but a conservative deployment should use refusal as the fallback.
- **Simple policy family.** One scalar level shared across models is easy to certify but not optimal, and the stylised simulator controller is not the shipped one; the real-trace studies use the shipped admission rule but no model routing, because routing changes the trajectory and cannot be replayed.
- **Workloads.** Two real task families, one provider's prices, and re-priced coding-agent costs. The real studies sit at the two extremes of κ (about 70 and about 0.02), and the short-horizon runs have 2.6 calls; a workload with long outputs, many calls and tight budgets would test the middle of the range.
- **Power at tight tolerances.** Validity holds at any n, but at $\varepsilon \leq 0.02$ 1,000 runs can leave the certified policy up to 21 points of capable-model use below the best safe member of the family.
- **Mode B** assumes exact prompt counts; with an under-counting estimator the backstop `max_tokens` can still let a call overrun.

- **Early stopping depends on its predictor.** The certificate holds for any predictor fitted on separate data, but the saving does not: with the features in these logs, runs become predictable only after 15–30 calls, and stopping at the start of a run gives little benefit. The stopping rule was evaluated on one agent and one model's trajectories, re-priced. In deployment the library's features use the router's own prompt estimate, which must match the counts the model was fitted on, and one router must serve one run.
- **Completion under output reservation is not controlled.** Certified reservation policies can refuse more runs than expected-cost reservation; the loss "overrun or incomplete" would control both.

12 Conclusion

Run-level budget risk can be certified from i.i.d. runs alone, at a cost of a few hundred to a few thousand logged runs, without modelling output lengths or the dependence between steps, and the certificate held in every setting we tested. How much it buys depends on two things. The ratio κ decides whether output reservation matters at all: when one maximum-length output is a large share of the budget, the certified policy recovers capability that worst-case reservation and per-step bounds forfeit; when it is a small share, overspending is rare under any policy. And the tolerance decides how much the run-level search adds over simply certifying expected-cost reservation: little at $\varepsilon = 0.05$ on our real workload, a great deal at $\varepsilon \leq 0.02$. For long-horizon agents the budget's main cost is runs that cannot finish within it, and the same certification lets an operator stop them early with a guaranteed bound on the share of runs stopped although they would have solved their task; on real coding-agent runs, with that share at most 5%, this saved 11–23% of spend.

Reproducibility. All code, logged runs and analysis outputs are in the `research/` directory of `https://github.com/orvi2014/Baar-Core` at commit

`d5ff5dbdf98a66f7c9a8e6bc6f64ed49d9995272`

and the library methods are in releases v0.11.0, v0.12.1 and v0.13.0. Every table is generated from the analysis outputs by `research/sim/make_tex.py` and `make_tex_es.py`.

Competing interests. The author maintains Baar-Core, the open-source library in which the methods are implemented.

AI usage. Generative AI tools (Claude Opus and Claude Sonnet, Anthropic, through Claude Code) were used to write simulation and analysis code and to draft and edit this paper. The author chose the research question and method, checked the proofs, reviewed all code and text, and is responsible for the content.

References

[1] Eitan Altman. *Constrained Markov Decision Processes*. Chapman and Hall/CRC, 1999.

[2] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control, 2021. arXiv:2110.01052.

[3] Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. Conformal risk control, 2022. arXiv:2208.02814.

[4] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents, 2025. arXiv:2505.20411.

[5] Longju Bai, Zhemin Huang, Xingyao Wang, Jiao Sun, Rada Mihalcea, Erik Brynjolfsson, Alex Pentland, and Jiaxin Pei. How do AI agents spend your money? analyzing and predicting token consumption in agentic coding tasks, 2026. arXiv:2604.22750.

[6] Stephen Bates, Anastasios Angelopoulos, Lihua Lei, Jitendra Malik, and Michael I. Jordan. Distribution-free, risk-controlling prediction sets, 2021. arXiv:2101.02703.

[7] Gaston Besanson. Green SARC: Predictive cost and carbon governance for agentic AI systems, 2026. arXiv:2606.15954.

[8] Marco C. Campi and Simone Garatti. The exact feasibility of randomized solutions of uncertain convex programs. *SIAM Journal on Optimization*, 19(3):1211–1230, 2008. doi: 10.1137/07069821X.

[9] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance, 2023. arXiv:2305.05176.

[10] Shai Feldman and Yaniv Romano. Spend where it counts: Dynamic budget allocation for multi-turn LLM evaluation, 2026. arXiv:2605.06605.

[11] Bror Hultberg, Dave Zachariah, and Antônio H. Ribeiro. Anytime-valid conformal risk control, 2026. arXiv:2602.04364.

[12] Sajjad Khan. Token budgets: An empirical catalog of 63 LLM-agent budget-overrun incidents, with an affine-typed Rust mitigation as a case study, 2026. arXiv:2606.04056.

[13] Yuxiang Lin, Zihan Wang, Mengyang Liu, Yuxuan Shan, Longju Bai, Junyao Zhang, Xing Jin, Boshan Chen, Jinyan Su, Xingyao Wang, Jiaxin Pei, and Manling Li. BAGEN: Are LLM agents budget-aware?, 2026. arXiv:2606.00198.

[14] Nebius. SWE-rebench-openhands-trajectories. Hugging Face dataset, revision 35455389ab51bf5e2306bfd436ef72d0f98bf882, CC-BY-4.0, 2025. URL `https://huggingface.co/datasets/nebius/SWE-rebench-openhands-trajectories`.

[15] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data, 2024. arXiv:2406.18665.

[16] Jeffery Opoku and David Banahene. ToolChain-CRC: Conformal risk control for agentic AI under retrieval and tool-use drift, 2026. arXiv:2606.18467.

[17] OWASP Foundation. OWASP top 10 for LLM applications 2025: LLM10 unbounded consumption, 2025. URL `https://genai.owasp.org/llmrisk/llm102025-unbounded-consumption/`.

[18] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2207.07061.

[19] Maria Trofimova, Anton Shevtsov, Ibragim Badertdinov, Konstantin Pyaev, Simon Karasik, and Alexander Golubev. OpenHands trajectories with Qwen3-Coder-480B-A35B-Instruct. Nebius blog, 2025. URL `https://nebius.com/blog/posts/openhands-trajectories-with-qwen3-coder-480b`.

[20] Antonios Valkanas, Soumyasundar Pal, Pavel Rumiantsev, Yingxue Zhang, and Mark Coates. C3PO: Optimized large language model cascades with probabilistic cost constraints for reasoning. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2511.07396.

[21] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, et al. OpenHands: An open platform for AI software developers as generalist agents, 2024. arXiv:2407.16741.

[22] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering, 2018. arXiv:1809.09600.

[23] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models, 2022. arXiv:2210.03629.